Automatic Validation of LLM-Generated Code with Prompt Paraphrasing

Xiaoyin Wang
The University of Texas at San Antonio
San Antonio, USA
xiaoyin.wang@utsa.edu

Dakai Zhu
The University of Texas at San Antonio
San Antonio, USA
dakai.zhu@utsa.edu

Abstract

Large Language Models (LLMs) have been widely used for code generation. However, the quality of generated code is still questionable, and code validation remains a challenging problem. In this paper, we propose a novel solution called paraphrase prompt testing. Our intuitive observation is that intrinsic consistency always exists among correct code pieces but may not exist among flawed code pieces, so we can detect flaws in the code by detecting inconsistencies. Therefore, we can vary a given prompt to multiple prompts with paraphrasing, and to ask the LLM to acquire multiple versions of generated code, so that we can validate whether the semantic relations still hold in the acquired code through cross-validation. Our initial evaluation on HumanEval shows that paraphrase prompt testing is able to detect 70.6% of the erroneous programs generated by GPT-4o, with a false positive rate of 6.1%.

Keywords

LLM, Code Validation, Prompt Paraphrasing

ACM Reference Format:
Xiaoyin Wang and Dakai Zhu. 2026. Automatic Validation of LLM-Generated Code with Prompt Paraphrasing. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-NIER '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3786582.3786809

1 Introduction

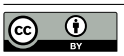
Large Language Models (LLM) have shown their capability of generating code based on natural language prompts [23] [25] [29]. It is reported [3] that GPT-4o can create correct programs for 89% of a set of 164 natural language prompts in HumanEval [6]. Such capability has the potential to largely reduce manual effort in software development, but also raises concern on the quality and correctness of LLM-generated code [16].

Existing techniques on model and code validation can hardly detect erroneous LLM-generated code pieces in practice. Some benchmarks (e.g., HumanEval [6]) and techniques (e.g., EvalPlus [18]) have been developed to evaluate the capability of LLMs on code generations, but they all rely on pre-defined canonical solutions (i.e., ground truth programs), so they can be used only for the evaluation of LLMs instead of validation of LLM-generated code, because canonical solutions do not exist in the latter scenario (otherwise LLM-based code generation would be unnecessary). A commonly used solution in other AI application areas is to have a human being to double check AI products, such as have a proofreader to double check text generated by AI models. However, in code generation, it often takes more effort for professional developers to understand and correct other developers' code (or AI-generated code) than to write the code piece themselves [33]. Automatic testing approaches [5] [21] [36] can automatically generate test inputs to test LLM-generated code, but due to the lack of test oracles , the generated tests can detect only runtime errors such as crashes instead of assertion errors where the program execution completes successfully but with an erroneous output. Our evaluation of GPT-4o [1] on HumanEval (see Section 4) shows that most run-time errors triggered by LLM-generated code are assertion errors.

In this paper, we propose *paraphrase prompt testing*, a novel solution based on metamorphic testing [7] to validate LLM-generated code. Metamorphic testing takes advantage of the known relations between certain inputs and outputs to test programs without test oracles. The relations been used in metamorphic testing are called $metamorphic relations$, and one biggest challenge of adopting metamorphic in general testing [20] [34] is to find a general metamorphic relations working for all code.

In *paraphrase prompt testing*, instead of leveraging concrete metamorphic relations, we consider just one simple metamorphic relation: *Taking prompts with the same meaning, the LLM should generate programs with the same semantics.*. Given a prompt, we first use NLP-based paraphrasing [31] [28] [30] to create multiple paraphrases of the prompt, referred to as *paraphrase prompts*. Second, we feed the original prompt to the LLM-based code generator and generate the original LLM-generated program. This program is the one we would like to validate and we refer to it as the *target program*. Third, we feed all paraphrase prompts to the LLM-based code generator and generate a set of LLM-generated programs which we referred to as the *paraphrase programs*. Fourth, we use the automatic test generation technique [5] to create random inputs to feed into the target program as well as the paraphrase programs and check whether the output of the target program is identical with those from the majority of paraphrase programs.

Our initial evaluation of paraphrase prompt testing is on the well known benchmark HumanEval [7]. Our experiment results show that, among 164 code generation prompts in HumanEval, our approach is able to detect 70.6% (potentially 76.5% with a more conservative configuration) of the 17 erroneously generated programs with a false positive rate of 6.1%. In other words, a developer needs to look at only 21 LLM-generated programs to find 12 of them as truly erroneous (with 9 false positives), while missing only 5 erroneous programs.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-NIER '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2425-1/26/04
https://doi.org/10.1145/3786582.3786809

2 Related Works

AI-based code generation starts from early efforts on API method suggestion [14] and code auto-completion [9] based on traditional machine learning models [19] and later deep neural network [9]. More recently, researchers start to train large code models [13] [22], or incorporating large amount of code when training general-purpose LLMs [1]. This not only accelerates the coding process but also helps in reducing code errors [37] [24] [35], leading to more efficient and reliable software development.

Metamorphic Testing [7, 12] overcomes the test oracle problem by leveraging metamorphic relations , so it can test software with non-deterministic output [2, 32, 38]. More recently, Ma et al. [20], Hyun et al. [15] and Cho et al. [8] developed metamorphic-testing-based approach to test NLP models and LLMs. In compiler validation, Equivalence Modulo Inputs [10, 17] uses semantically equivalent program mutants to test compilers. Compared with these techniques, we propose a novel generic metamorphic relation that can be directly used to test all LLM-generated code.

3 Approach

The overview of our approach is illustrated in Figure 1. Our approach takes just a prompt as the input and does not need any additional information. The prompt is fed into (1) the prompt paraphrasing component to generate paraphrase prompts, and (2) the LLM to generate a target program. The paraphrase prompts are also fed into the LLM to create paraphrase programs. After that, we apply automatic testing to the target program to create a set of test inputs (without test oracles). The test inputs are then executed on the target program and paraphrase programs for cross validation.

Listing 1: Paraphrases of Prompt No. 9

```
from typing import List, Tuple
def rolling_max(numbers: List[int]) -> List[int]:
    """
From a given list of integers, generate a list
of rolling maximum element found until given
moment in the sequence.
>>> rolling_max([1, 2, 3, 2, 3, 4, 2])
    [1, 2, 3, 3, 3, 4, 4]
    """
-----------------------------------
from typing import List, Tuple
def rolling_max(numbers: List[int]) -> List[int]:
    """
Create a list that details the highest integer
present up to each point within a provided
sequence of integers.
>>> rolling_max([1, 2, 3, 2, 3, 4, 2])
    [1, 2, 3, 3, 3, 4, 4]
    """
```

3.1 Generation of Paraphrase Programs

We illustrate how we generate paraphrase programs using List 1. The first section (above the separation line) shows an original prompt in HumanEval, which consists of the following parts.

- The import statements declaring the libraries to be used, as shown in the first line of each section in List 1.
- The signature of the function to be implemented, including the types of inputs of outputs, as shown in the second line of each section in List 1.
- The natural language description of the function to be implemented, as shown in Lines 4-6 of each section.
- Examples showing sample inputs and corresponding outputs, as shown in Lines 7 and 8 of each section of List 1.

Since we need to run all the paraphrase programs with the same set of test inputs, we need to make sure that they all have the same signature. The examples also need to be left identical because any changes on them will lead to a semantic change. Therefore, when creating paraphrases of prompts, we consider only the description part of a prompt, while leaving all other parts untouched. To create paraphrase of the description part, we use the same LLM model, and feed a prompt by adding "Can you paraphrase the following paragraph?" before the description part. Once paraphrases are provided, we replace the original description part with the paraphrases to create the paraphrase prompts as shown in the second section of List 1. Finally, we use the original prompt and the paraphrase prompts to generate the target program and the paraphrase programs.

Algorithm 1 Cross Validation Algorithm

Input: Target, Paraprogs, Tests
Output: ErrorFlag

```
1:  Let ErrorFlag = False
2:  Let DiffSet = ∅
3:  for Test ∈ Tests do
4:      Let Output = run(Target, Test)
5:      for Para ∈ Paraprogs do
6:          if run(Para, Test)! = Output then
7:              Add Para to DiffSet
8:          end if
9:      end for
10: end for
11: return len(DiffSet) > len(Paraprogs)/2
```

3.2 Automatic Test Generation

Since our main goal is to validate the target program, we want the automatically generated test inputs exercise most of its paths and achieve high code coverage. Therefore, we consider only the target program when generating test inputs.

Conceptually, our approach can adopt any automatic test generation tool, so it can be enhanced with enhancements in test generation. In our implementation, we use the python-AFL tool [27] (python-oriented version of AFL [11]) to create test inputs because it is very robust and uses the state-of-the-practice coverage-guided fuzzing technique. In particular, it monitors the program to see which parts of the code are exercised, and then uses the information to generate new test input through mutation that targets unexplored areas of the program.

3.3 The Cross Validation Algorithm

After we acquire the set of test inputs, we will run the target program and the paraphrase programs and cross validate the results. A most conservative strategy would be reporting an error whenever a

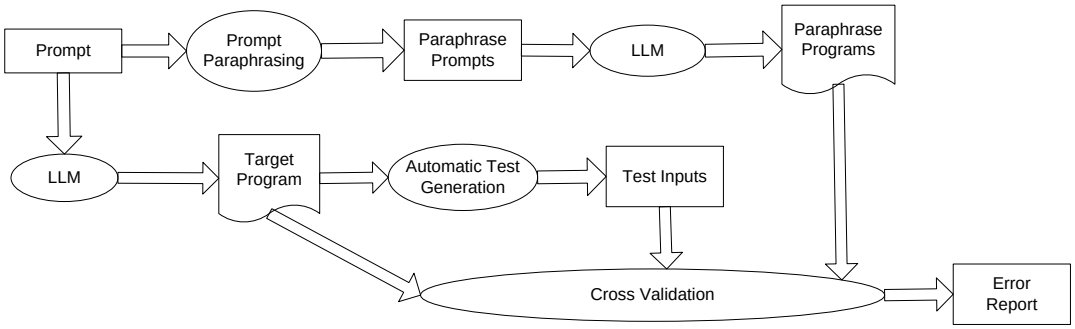


Figure 1: Approach Overview

different output is seen (i.e., when any paraphrase program's output on any input is different from that of the target program). While this can be considered as a solution for for critical usage scenarios, we believe such strict consistency may not be necessary given that noises may be brought in during the paraphrase generation process (e.g, an erroneous paraphrase may have a different meaning from the original prompt).

To achieve a more balanced cross validation, we developed Algorithm 1. The basic idea is to report an error only if the majority of the paraphrase programs have different semantics from the target program (i.e., generating a different output for any test input). The algorithm takes three inputs: the target program ($Target$), the set of paraphrase programs ($Paraprogs$), and the set of test inputs ($Tests$). In the algorithm, we execute each paraphrase programs with each input (Line 6) and compare the results with output of the target program. If there is any difference, we will add the corresponding paraphrase program into the $DiffSet$. Finally, we check whether the size of $DiffSet$ is more than half of the size of paraphrase programs.

4 Initial Experiments

In this section, we will introduce the initial evaluation results of our approach on the HumanEval benchmark.

4.1 Research Questions

- **RQ1:** How effective is our approach on detecting erroneous LLM-generated code?
- **RQ2:** How our approach compares with its variants on detecting erroneous LLM-generated code?
- **RQ3:** Why our approach fails to detect some erroneous LLM-generated code?

4.2 Evaluation Setup

In our initial evaluation, we use the widely adopted HumanEval dataset [6]. The dataset consists of 164 pairs of human written prompts and corresponding canonical solutions as ground truth. It also provides a set of test cases for each prompt so that a given LLM-generated program can be evaluated against the canonical solution. It should be noted that we use neither canonical solutions nor test cases in our approach. We use the `gpt-4o-2024-0806` model provided by OpenAI as the LLM model, because it is the state-of-the-art model for code generation [3], and the model we use is its most recent stable version.

#prompts	Accuracy	Recall	Precision	FP Rate
3	90.2%	52.9%	52.9%	5.4%
5	91.5%	70.6%	57.1%	6.1%
7	90.9%	70.6%	54.5%	6.8%

Table 1: Our Effectiveness with Different Number of Prompts

4.3 Experiment Results

To answer **RQ1**, for each prompt, we check the LLM-generated target program against the canonical solution with test cases provided by HumanEval to acquire the ground truth label of the target program. Then we compare the reports of our approach with the ground truth. The results are presented in Table 1.

In the table, Column 1 shows the number of prompts we used. We consider odd numbers only here so that our cross validation always works with a clear majority. Columns 2-5 present the accuracy, recall, precision, and FP rate, respectively. From the table, we have the following observations. First of all, the best configuration of our approach (#prompts = 5) is able to averagely detect 12 (70.6%) out of all 17 erroneous programs generated by GPT-4o, with a false positive rate of 6.1%.

In other words, a developer need to look at only 21 LLM-generated programs to find 12 of them as truly erroneous (with 9 false positives), while missing only 5 erroneous programs. Second, the configuration with 3 prompts has much lower recall and lower false positive rate as well. When the number of prompts is small, due to the lower variety of prompts, the approach may have less power to create different paraphrase program variants, making it more difficult to detect erroneous programs. Third, the configuration with 7 prompts achieves the same recall as the configuration with 5 prompts, but with more false positives, so its precision and FP rate are lower. The reason behind may be that more prompts may lead to more noises in the paraphrase generation process (i.e. paraphrases generated by LLM may also not be true paraphrases).

To answer **RQ2**, we perform an ablation study to find out whether our prompt paraphrasing technique is effective. So, we compare our approach with an alternative that does not perform paraphrasing, but just feed the same prompt to the LLM for multiple times. Since we use five prompts as the default configuration, we also feed the same prompt five times in this variant. Another technique we want to evaluate is our cross-validation algorithm. Here in the variant we consider the more conservative cross validation, where we report an error whenever we see a different output from any paraphrase program on any input.

Variant	Accuracy	Recall	Precision	FP Rate
Our Approach	91.5%	70.6%	57.1%	6.1%
w/o Paraphra.	90.2%	47.1%	53.3%	4.8%
conservative cross valid.	84.1%	76.5%	37.1%	15.0%

Table 2: Effectiveness of Our Approach with Different Techniques

From Table 2, we have the following observations. First, compared with our approach, the variant without paraphrasing achieves lower false positive rate, but its recall is also much lower. Since repetitively feeding in the same prompt may not trigger more variance in the code generation process, the newly generated code is more likely to be identical or very similar to the target program. Therefore, this variant is not as effective as our approach on detecting erroneous LLM-generated program. Second, the variant using conservative cross validation achieves slightly higher recall but its precision is much lower. In particular, the precision drops to 37.1%, so that a developer needs to review 35 programs (14 more programs than our default setting) to detect one more true erroneous programs. However, the additional effort may worth it in critical scenarios because this variant does find one more erroneously generated program. Furthermore, the false positive rate of 15.0%, although higher than all other variants and configurations of our approach, still indicate that 85% of the correct programs have been filtered out so a developer may not need to review them.

4.4 Qualitative Analysis

To answer **RQ3**, we manually analyzed the failing cases (false positives and false negatives) of our approach. Our manual analysis shows that the major reason for false negatives of our approach lies in the high similarity among paraphrases, as shown in List 2. Note that we show only two paraphrase prompts here for briefness.

Listing 2: Paraphrases of Prompt No. 83

```
def starts_one_ends(n):
"""
For a given positive integer n, calculate the
total quantity of positive integers with n digits
that either begin or conclude with the digit 1.
"""
------------------------------------
def starts_one_ends(n):
"""
For a given positive integer n, calculate the
total amount of n-digit positive integers that
either begin or conclude with the digit 1.
"""
------------------------------------
def starts_one_ends(n):
"""
For a positive integer n, calculate the total
number of n-digit positive integers that either
begin or conclude with the digit 1.
"""
```

From the List, we can see that the two paraphrase prompts generated are very similar to the original prompt. The lack of variety in paraphrase prompts caused the LLM to generate very similar paraphrase programs and thus all of them share the same wrong semantics.

Our manual analysis of false positives show that the major reason lies in the paraphrase clustering effect, where paraphrase prompts are different from the original prompt, but the similarity among the paraphrase prompts themselves are high. Since all the paraphrase prompts look very similar to each other, once the LLM generate erroneous code for one of them, it will also generate erroneous code for all of them, and thus these erroneous code easily holds a majority among prompts and cause our approach to report a false positive. From these observations, we can see that the similarity among generated paraphrase prompts plays an important role in the effectiveness of our approach. Therefore, it is possible that we can measure the similarity to indicate whether our approach is applicable or not in certain cases. It may also be possible to try to enlarge the variance of generated paraphrase prompts to further enhance our approach.

5 Future Plans

Our results show that paraphrase prompt testing is a promising approach to validate LLM-generated code. We plan to further extend our research towards a full paper on the following aspects.

- **Leveraging Paraphrase Similarity.** Inspired by our qualitative study, we plan to investigate how similarity among paraphrase prompts may affect the effectiveness of our approach. We will also develop techniques taking similarity into consideration. For example, we can generate more paraphrase prompts and perform similarity filtering to use only paraphrase prompts within certain similarity range. The optimal similarity range would need to be determined through experiments. Another way to rule out bad paraphrase prompts is to direct ask a different LLM model whether a generated paraphrase prompt has the same meaning with the original prompt.
- **Considering Multiple LLMs**. our approach now leverages the GPT-4o model for both paraphrase and code generation. Note that using a single model for both tasks is not a threat to our evaluation because the target programs are finally evaluated with only canonical solutions. In contrast, achieving good results with a single model shows that our approach has potential to be further improved when using more than one LLM models. For example, we can use Gemini to generate paraphrase prompts and use them to cross validate code generated by GPT-4o, and vice versa. We plan to use multiple LLM models to enhance our approach and generalize our evaluation results.
- **More Code Generation Benchmarks.** We plan to extend the scope of our experiments to more benchmarks such as HumanEval-XL [26] and MBPP [4].

Our idea of prompt paraphrasing also opens up a whole new route toward LLM-generated code validation, based on intentional variance generation and cross validation. For example, a natural following work from the opposite direction is to apply semantic preserving code changes to LLM-generated code and use LLMs to report whether the changed programs perform the same task in the original prompt.

This work is supported in part by NSF grants CCF-1846467.

References

[1] Gerardo Adesso. 2022. GPT4: The ultimate brain. *Authorea Preprints* (2022).

[2] John Ahlgren, Maria Berezin, Kinga Bojarczuk, Elena Dulskyte, Inna Dvortsova, Johann George, Natalija Gucevska, Mark Harman, Maria Lomeli, Erik Meijer, et al. 2021. Testing web enabled simulation at scale using metamorphic testing. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 140–149.

[3] AI Anthropic. 2024. Claude 3.5 sonnet model card addendum. *Claude-3.5 Model Card* 3 (2024), 1–8.

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[5] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2329–2344.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–27.

[8] Steven Cho, Stefano Ruberto, and Valerio Terragni. 2025. Metamorphic testing of large language models for natural language processing. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 174–186.

[9] Juan Cruz-Benito, Sanjay Vishwakarma, Francisco Martin-Fernandez, and Ismael Faro. 2021. Automated source code generation and auto-completion using deep learning: Comparing and discussing current language model-related approaches. *AI* 2, 1 (2021), 1–16.

[10] Alastair F Donaldson and Andrei Lascu. 2016. Metamorphic testing for (graphics) compilers. In *Proceedings of the 1st international workshop on metamorphic testing*. 44–47.

[11] Andrea Fioraldi, Alessandro Mantovani, Dominik Maier, and Davide Balzarotti. 2023. Dissecting American fuzzy lop: a FuzzBench evaluation. *ACM Transactions on Software Engineering and Methodology* 32, 2 (2023), 1–26.

[12] Ralph Guderlei and Johannes Mayer. 2007. Statistical metamorphic testing testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In *Seventh International Conference on Quality Software (QSIC 2007)*. IEEE, 404–409.

[13] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.

[14] Qiao Huang, Xin Xia, Zhenchang Xing, David Lo, and Xinyu Wang. 2018. API method recommendation without worrying about the task-API knowledge gap. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 293–304.

[15] Sangwon Hyun, Mingyu Guo, and M Ali Babar. 2024. METAL: Metamorphic Testing Framework for Analyzing Large-Language Model Qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128.

[16] Sadia Jahan and Xiaoyin Wang. 2025. How Does ChatGPT Make Assumptions When Creating Erroneous Programs?. In *International Conference on Automated Software Engineering (NIER Track)*. IEEE.

[17] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. *ACM Sigplan Notices* 49, 6 (2014), 216–226.

[18] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210* (2023).

[19] Xiaoyu Liu, LiGuo Huang, and Vincent Ng. 2018. Effective API recommendation without historical software repositories. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 282–292.

[20] Pingchuan Ma, Shuai Wang, and Jin Liu. 2020. Metamorphic Testing and Certified Mitigation of Fairness Violations in NLP Models.. In *IJCAI*. 458–465.

[21] Phil McMinn. 2011. Search-based software testing: Past, present and future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. IEEE, 153–163.

[22] Anh Tuan Nguyen, Trong Duc Nguyen, Hung Dang Phan, and Tien N Nguyen. 2018. A deep neural network language model with contexts for source code. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 323–334.

[23] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*. PMLR, 26106–26128.

[24] Xintao Niu, Changhai Nie, Hareton Leung, Yu Lei, Xiaoyin Wang, Jiaxi Xu, and Yan Wang. 2018. An interleaving approach to combinatorial testing and failure-inducing interaction identification. *IEEE Transactions on Software Engineering* 46, 6 (2018), 584–615.

[25] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. 2023. LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation. *arXiv preprint arXiv:2308.02828* (2023).

[26] Qiwei Peng, Yekun Chai, and Xuhong Li. 2024. Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization. *arXiv preprint arXiv:2402.16694* (2024).

[27] Python-AFL. [n. d.]. Python-AFL. https://github.com/jwilk/python-afl.

[28] Hassan Shahmohammadi, MirHossein Dezfoulian, and Muharram Mansoorizadeh. 2021. Paraphrase detection using LSTM networks and handcrafted features. *Multimedia Tools and Applications* 80 (2021), 6479–6492.

[29] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.

[30] Xiaoyin Wang, David Lo, Jing Jiang, Lu Zhang, and Hong Mei. 2009. Extracting paraphrases of technical terms from noisy parallel software corpora. In *Proceedings of the ACL-IJCNLP 2009 Conference Short Papers*. 197–200.

[31] Sam Witteveen and Martin Andrews. 2019. Paraphrasing with large language models. *arXiv preprint arXiv:1911.09661* (2019).

[32] Chaohua Wu, Liqun Sun, and Zhi Quan Zhou. 2019. The impact of a dot: Case studies of a noise metamorphic relation pattern. In *2019 IEEE/ACM 4th International Workshop on Metamorphic Testing (MET)*. IEEE, 17–23.

[33] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E Hassan, and Shanping Li. 2017. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2017), 951–976.

[34] Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Unveiling hidden dnn defects with decision-based metamorphic testing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[35] Xueling Zhang, John Heaps, Rocky Slavin, Jianwei Niu, Travis Breaux, and Xiaoyin Wang. 2023. Daisy: Dynamic-analysis-induced source discovery for sensitive data. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–34.

[36] Xueling Zhang, Xiaoyin Wang, Rocky Slavin, and Jianwei Niu. 2021. Condysta: Context-aware dynamic supplement to static taint analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 796–812.

[37] Xiaoyu Zhang, Juan Zhai, Shiqing Ma, and Chao Shen. 2021. Autotrainer: An automatic dnn training problem detection and repair system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 359–371.

[38] Zhi Quan Zhou, Liqun Sun, Tsong Yueh Chen, and Dave Towey. 2018. Metamorphic relations for enhancing system understanding and use. *IEEE Transactions on Software Engineering* 46, 10 (2018), 1120–1154.